

RL Closed Loop Autonomous Vehicle Training in Simulation for High Interactive Scenes

Ishan Gupta, Leo Shaw, Linjie Zhang

Carnegie Mellon University

Pittsburgh, PA

ishang@andrew.cmu.edu, leos@andrew.cmu.edu, linjiez@andrew.cmu.edu

Abstract—Autonomous vehicles are still struggling in highly interactive traffic situations such as unprotected left turns, highway merges, and sudden cut-ins. This paper presents a closed-loop simulation framework in CARLA that studies these scenarios in a controlled environment. We first establish a classical baseline using global route planning, local planning, and a fixed low-level controller. This method is functional, but limited by the reactive planning behavior in difficult interactions. We then extend the framework with a reinforcement learning planner that produces short-horizon trajectories while keeping the same low-level controller fixed, allowing planning improvements to be evaluated directly. In the 100-episode highway-merger evaluation summarized in our repository, the PPO planner achieves an 86 percent success rate, with a 90th-percentile successful completion time of 17.4 s and a 90th-percentile maximum jerk of 482.0 m/s^3 . Compared with the aggressive behavior-agent baseline, PPO is already functional in closed loop and can complete many interactive merges, but it still trails the baseline in success rate, efficiency, and comfort. These results provide both a reproducible baseline and a practical foundation for improving interactive motion planning in autonomous driving.

Index Terms—Motion Planning, Autonomous Driving, CARLA Simulator, PPO, Reinforcement Learning

I. INTRODUCTION

Autonomous vehicles should be safe, smooth, and efficient in reaching their destination, while interacting with other agents in complex urban and highway settings. These settings are challenging, since driving decisions depend on multiple factors, especially in a need to react to the behavior of many other actors in high-conflict regions. Even strong modern learning-based stacks still report that collisions frequently arise during unprotected turns and that merging/lane changes in dense traffic remain difficult [1]. These issues motivate studying trajectory generation in complex scenarios (such as unprotected left turns, highway merges, and abrupt cut-ins) and doing so in closed-loop simulation where critical outcomes can be observed repeatedly under controlled variation [2].

Early autonomous driving systems such as Boss showed that autonomous vehicles are possible with carefully created modular pipelines that break down this task into perception, planning, and control [3]. Later work further broke the pipeline down into route planning, behavioral decision-making, motion planning, and feedback control, which remains a standard way to reason about the autonomy stack [4]. With the introduction of world simulation tools such as CARLA, the field was able to study difficult situations that were too dangerous to explore

in the real world. With a repeatable environment, learning-based approaches became feasible, and moved from modular rule-based planning toward imitation and end-to-end policies.

In this work, we present a scenario-driven simulation pipeline in CARLA for studying closed-loop trajectory planning under interactive traffic, focusing on unprotected left turns, highway merging, and abrupt cut-ins. We chose unprotected left turn because it is a high-conflict urban maneuver where the ego vehicle must decide whether to yield or commit while reasoning about oncoming traffic and limited acceptable gaps. Left turns are also widely recognized as a higher-risk interaction than straight or right-turn movements as the vehicle is reasoning against on-coming traffic. We chose highway merge because it tests a different way of interaction where the ego vehicle must coordinate longitudinal speed adjustment and lateral insertion into ongoing traffic under limited space and timing. Lastly, abrupt cut-ins test the robustness and response of the autonomous vehicle to unpredictable situations. Together, these scenarios address common interaction-high events that are commonly scene on the road. Our approach focuses on a learner that will produce a short-horizon planned motion (trajectory or waypoint sequence) which is executed by a fixed low-level PID tracker, enabling us to attribute performance differences primarily to the planner rather than to a changing controller. This paper reports our Phase 2 foundation: (i) a reproducible CARLA setup and baseline implementation (A* route planning, waypoint queue local planning, and PID tracking) and (ii) the implementation of PPO for both the highway merge and left turn scenario.

II. RELATED WORK

Our review focuses on studies that highlight classical local trajectory planning stacks that are commonly used as simulation baselines, planning with interaction and uncertainty at intersections/merges, and learning-based trajectory models using waypoint planning with separate tracking.

A. Classic Local Trajectory Planning

In industry, the planning module and the control module are generally broken down into route planning, a behavioral layer, motion planning, and a local feedback controller. Paden et al. describes this hierarchy and explains that motion planning produces a set of points that are then followed by a feedback controller to correct tracking error. The set of points can be

a path that specifies geometry without timing, or a trajectory which is time-parameterized over a horizon [4]. Instead of combining the planning and controls in one algorithm, we use the defined hierarchy in our project pipeline by treating the planner output as a short-horizon trajectory and using a separate controller. This way, we can attribute any improvements to the planning layer rather than to changes in low-level control.

The CARLA modular pipeline follows this pipeline using a waypoint-based local planner and a classic PID controller. The local planner generates near-term goal states (waypoints) that indicate the position and orientation of the car at different time steps (trajectory). The trajectory is then used in a PID controller that actuates the vehicle’s steering/throttle/brake [2]. Modern CARLA learning papers also build models and closed-loop baselines around this interface. For example, TransFuser’s expert policy consists of an A* planner followed by two PID controllers. The TransFuser model itself predicts short-horizon waypoints and uses the two controllers to convert waypoints into low-level commands [1]. Regarding low-level control, PID tracking quality can be improved through more advanced control algorithms like MPC as comparison studies have shown [5]. In Phase 1, our baselines and planned learning systems will use a fixed tracker (PID) and focus on planning outputs (waypoints/ trajectory), hence aligning our evaluation with the pipeline seen in CARLA. This will also isolate the effect of the planning module if the controller is kept constant.

B. Interaction and uncertainty at intersections/merges

Intersections are locations where complexity increases as the number of agents, rules, and interactions increase. These complex locations are more prone to collisions compared to many other road segments. Wei et al. highlights that intersections have many conflict points and reports that left turns are more likely to collide than straight or right turns [6]. Furthermore, they classify “ramp merges” as an intersection type. This conclusion supports our Phase 1 focus on unprotected left turns and cut-ins/merges where reactive behavior is likely to be stressed.

One main difficulty in these scenarios is the partial observability and uncertainty of other agent’s intentions. A solution to this problem is to use probabilistic decision-making frameworks such as Markov Decision Process (MDP) and Partially Observed Markov Decision Process (POMDP) [4]. Hubmann et al. investigates a POMDP-based behavior planner that predicts the future field of view (FOV) while considering the possibility of an occluded vehicle. They show that the model has reduced conservatism compared to a baseline that only considers the current FOV [7]. Additionally, for an intersection scenario, Qiao et al. explains that TTC-style rule logic is difficult to make reliable due to changing conditions and motivates reinforcement learning alternatives for better behavior. They use a Hierarchical Options Markov Decision Process (HOMDP) to combat the computation inefficiencies of POMDP. Their results indicate there are higher collision rates for TTC relative to learned policies in the intersection scenario [8].

Both of these papers encourage planners that reason over future interactions and uncertainty rather than reacting to instantaneous gaps. Our project aims to react to these instantaneous gaps by learning short-horizon trajectories for interactive merges/turns and by explicitly planning to test robustness under observation noise and scenario variation.

In addition to interaction modeling, trajectory generation for lane-change maneuvers has been widely studied. Several works propose smooth polynomial or time-parameterized lateral trajectories to ensure comfort and feasibility during lane transitions. Zhang et al. [9] propose a lane-change trajectory planning method that uses smooth lateral displacement profiles to ensure geometric continuity and bounded acceleration during lane transitions, motivated by comfort and safety requirements in medium-to-high speed scenarios. Motivated by the smooth lateral trajectory formulation in [9], we implement a cosine-based easing function for NPC lane changes, with lateral offsets computed directly from waypoint transforms rather than assuming fixed lane-width spacing. However, unlike [9], which focuses on full trajectory planning and tracking for the autonomous vehicle using an APF-MPC framework, our Phase 1 study isolates the longitudinal response of a classical PID controller under controlled cut-in conditions. This separation allows us to evaluate baseline control robustness independent of learning-based decision modules.

C. Learning-based trajectory/waypoint planning with separate tracking

Rather than learning a fully end-to-end control or sticking with current hand-engineered stacks, we can aim for a mid-to-mid control pipeline: Predict the trajectory using a learned model and use a separate controller to execute the action. ChauffeurNet builds on this architecture by creating an imitation-learning model that produces a control trajectory. They note that standard behavior cloning (BC) is insufficient for complex driving scenarios even with a large data set. ChauffeurNet instead implements synthesized trajectories, and additional losses increase robustness. Additionally, they note that highly interactive scenarios like merges may need reinforcement learning (RL) framework for better results [10]. This directly relates to our project where we use the RL framework to better address interactions during left-turns, highway merges, and cut-ins.

Within CARLA, there are some learning systems that explicitly use waypoint prediction with PID tracking as the interface between learning and control. Learning by Cheating predicts waypoints with a BC-based approach and converts them into steering, twist, and brake using two independent PID controllers [11]. TransFuser similarly uses an autoregressive waypoint prediction module and then applies two PID controllers to produce low-level actuation. Despite strong performance, TransFuser reports that vehicle collisions remain challenging in dense traffic and observes that collisions occur mostly during unprotected turns and lane changes [1].

These results guide our chosen Phase 1 scenarios and justify our architectural approach. Similarly to Learning by Cheating

and TransFuser, we divide planning outputs from tracking, however, we pursue an RL-based framework to generate our trajectories rather than relying solely on imitation losses.

III. SYSTEM OVERVIEW

In CARLA’s baseline formulation, the agent follows a high-level planner, A*, which produces discrete navigation commands (e.g., turn left/right / straight/brake) but does not provide a geometric trajectory. A local planner then generates near-term waypoints using a rule-based state machine, and a PID controller executes steering/throttle/brake to track those waypoints. This waypoint to PID interface is also widely used in learning-based driving systems, where a network predicts waypoints and a low-level PID controller converts them into actuation commands (Figure 1) [1][11].

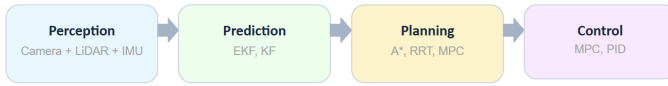


Fig. 1. Automation Pipeline: Shows the typical breakdown of autonomous system into four modules.

Although practical and widely adopted, rule-based planners coupled with PID tracking struggle in interactive settings because the planning logic is often reactive and is not optimized for safety under uncertain interactions. In CARLA’s modular pipeline, local planning is implemented as simple predefined policies in a rule-based state machine [2]. In dense traffic, learning papers also indicate increased collisions during unprotected turns and merging/lane changes. They suggest a better global contextual reasoning and/or a more effective interaction modeling for better results [1]. These observations suggest that a planner that learns to produce short-horizon trajectories and can later be stress-tested under sensing imperfections may better address the failure modes that appear in our target scenarios.

However, modern papers still report failures in dense interactive scenes such as unprotected turns, lane changes, and merges. That leaves a practical gap: how to improve short-horizon interaction planning in closed loop, while keeping the low-level controller fixed so planner improvements can be isolated clearly. That framing matches your current architecture and report direction.

IV. BASELINE

The objective of this study is to learn an RL model for three different scenarios (unprotected left turn, highway merge, and abrupt cut-in) that would take in perception data and output a trajectory. For phase 1, our goal was to set up the environment and baselines. Across all scenes, we use a rule-based CARLA baseline planner and a PID tracker as the reference system: a global route planner (A*), a local planner that maintains a queue of waypoints, and a low-level controller with longitudinal PID and lateral PID. The baseline has two profiles: Normal and Aggressive.

To quantify the difference between the two behavior profiles, we summarize their key parameter settings in Table I. These parameters define how aggressively the ego vehicle reacts to surrounding traffic.

TABLE I
COMPARISON OF NORMAL AND AGGRESSIVE BEHAVIOR PARAMETERS

Parameter	Normal	Aggressive
Max Speed (km/h)	50	70
Safety Distance (m)	10	8
Braking Distance (m)	5	4
Speed Limit Buffer	3	1
PID Controller	Same	Same

As shown in Table I, the aggressive profile operates with a higher maximum speed (70 km/h compared to 50 km/h in the normal profile), a reduced safety distance (8 m vs. 10 m), and a shorter braking distance (4 m vs. 5 m). These differences result in later braking and smaller accepted gaps when interacting with other vehicles.

In contrast, the normal profile maintains larger safety margins and reacts earlier to potential conflicts, leading to more conservative and stable driving behavior. Because both profiles use the same PID controller, these differences are entirely due to decision-level behavior rather than differences in low-level control.

The same scenario runner, traffic logic, and logging code is reused for both unprotected left turns and highway merges. The only changes are the map region, spawn points, and ego destination.

A. Traffic Imitation

Rather than attempting to imitate a specific real-world dataset, we imitate traffic phenomena relevant to each scene:

- Gap variability: randomized headways near the conflict zone (intersection) or merge zone (on-ramp to highway).
- Speed variability: randomized speed offset per vehicle to create heterogeneous flows.
- Density variability: number of active vehicles and spawn spacing.

This allows for simulation repeatability and for increasing difficulty through controlled parameter sweeps.

B. Scenario 1: Unprotected Left Turn

This scenario evaluates the ego vehicle’s ability to execute an unprotected left turn across oncoming traffic. The interaction-critical decision is when to commit to the turn given oncoming vehicles and limited acceptable gaps. The basic simulation flow is the ego approaches the intersection and slows down in the conflict zone. Then the baseline estimates a safe gap and decides whether to yield or go.

1) *Environment Selection*: We select a map where there is a T junction or a four-way intersection and no stop signs. In CARLA, map 01 has a T-junction with a traffic light. The light is set on green permanently to allow for free traffic flow.

2) *Simulation Setup*: The ego spawns on the approach lane and receives a route plan through the intersection that requires a left turn. The baseline agent uses A* for global routing and a local waypoint queue for short-horizon guidance. We spawn oncoming NPC traffic on the opposing lanes with routes that pass through the same conflict zone.

C. Scenario 2: Highway Merge

This scenario evaluates the ability of the ego vehicle to merge on a highway into flowing traffic. The interaction-critical decision is similar to the left-turn scenario where the vehicle must determine when to yield or merge into the traffic. The basic simulation flow is the ego approaches the merge, slows down, then merges in and keeps a safe distance from the other vehicles.

1) *Environment Selection*: Map 04 in CARLA provided the best environment where we had access to a merging road into a highway. The merge segment is short, hence forcing the ego vehicle to make a faster decision or abnormally slow down during the merge.

2) *Simulation Setup*: The ego spawns in the ramp/auxiliary lane and is assigned a route that requires merging into the rightmost highway lane before reaching a downstream goal. The baseline agent again uses global route planning and local waypoint tracking, executed by PID.

D. Simulation Run

For both scenario 1 and scenario 2, the same simulation execution logic was used. Each simulation run is executed as a sequence of episodes (epochs). We run repeated episodes because traffic initializations vary and interaction outcomes depend on timing. Following standard practice in simulation evaluations, we report success/collision rates across many trials rather than single-rollout anecdotes. Each episode loop, the world is reset to a known state, then the NPCs and ego vehicle are spawned in. The traffic is given some time to "warm up" to get some traffic flow and then the ego vehicle steps through the simulation till:

- the goal is reached
- collision terminates the episode
- maximum time horizon is exceeded

We log success/collision rates and safety/comfort proxies intended for later robustness comparisons (collisions, lane keeping, completion/speed, comfort tradeoffs).

E. Scenario 3: Sudden Cut-In

Objective: Demonstrate and analyze a closed-loop PID baseline model handling sudden highway cut-in maneuvers under controlled TTC-triggered interaction.

Setup

- Simulator: CARLA Town04 highway scenario
- Ego task: Maintain lane and safe forward progress
- Baseline model: TTC-gated interaction logic + smooth NPC lane-change trajectory + longitudinal PID controller
- Disturbance: Adjacent vehicle performs a single timed cut-in with configurable initial gap and speed

Scenario definition: The ego vehicle travels at a predefined target speed in the main lane, while an NPC vehicle is initialized in the adjacent lane with a configurable longitudinal offset. The NPC maintains constant speed until the longitudinal time-to-collision (TTC) between the two vehicles falls below a predefined threshold. Once triggered, the NPC executes a single smooth lane-change maneuver into the ego lane. Only one cut-in event is allowed per episode to ensure reproducibility and isolate controller behavior.

Basic simulation flow

- Ego travels at target speed under longitudinal PID control.
- At each simulation step, TTC between ego and NPC is computed using longitudinal gap and relative closing speed.
- When TTC falls below the threshold (`cut_at_ttc`), the NPC initiates a smooth lane-change trajectory.
- Ego reacts through throttle and brake commands to maintain safe longitudinal distance.

Cut-in triggering mechanism: The cut-in timing is determined using a longitudinal Time-to-Collision (TTC) formulation. Let $\mathbf{p}_e$ and $\mathbf{p}_n$ denote the ego and NPC positions, and $\mathbf{v}_e$ and $\mathbf{v}_n$ their corresponding velocity vectors. Let $\hat{\mathbf{f}}_e$ denote the unit forward direction of the ego vehicle.

The longitudinal gap is computed as

$$d = (\mathbf{p}_n - \mathbf{p}_e) \cdot \hat{\mathbf{f}}_e,$$

which represents the projected forward distance from the ego to the NPC.

The relative closing speed is defined as

$$v_r = (\mathbf{v}_e - \mathbf{v}_n) \cdot \hat{\mathbf{f}}_e,$$

which measures how fast the ego vehicle is approaching the NPC along the longitudinal direction.

For $v_r > 0$, the TTC is defined as

$$TTC = \frac{d}{v_r}.$$

If $v_r \leq 0$, the TTC is set to infinity, indicating that the ego vehicle is not approaching the NPC. The maneuver is triggered when $TTC < \text{cut_at_ttc}$. This formulation enables systematic control over interaction timing and scenario difficulty.

Lane-change modeling: To ensure physically consistent interaction dynamics, the NPC lateral motion is generated using a smooth cosine-based easing function defined along the longitudinal path of the current lane. Instead of assuming a fixed lane width, the true lateral offset between the source-lane center and the target-lane center is computed using waypoint transforms.

Let s denote the longitudinal distance traveled during the lane-change maneuver, and let $S = vT$ denote the total longitudinal distance corresponding to a lane-change duration T at speed v . The lateral displacement is defined as

$$d(s) = \frac{d_{\text{target}}}{2} \left(1 - \cos \left(\pi \frac{s}{S} \right) \right), \quad s \in [0, S],$$

where d_{target} is the signed lateral offset between the two lane centers.

At each step, the vehicle advances along the current lane centerline and applies the lateral offset $d(s)$ along the local right-vector direction of the waypoint. This formulation guarantees:

- Continuous lateral position
- Continuous lateral velocity
- Bounded lateral acceleration

Compared to abrupt lateral position updates, the cosine-based easing prevents non-physical jerk spikes and ensures that interaction dynamics remain realistic. Additionally, computing the true lane-center offset from waypoint transforms avoids over-shooting artifacts that may occur when assuming fixed lane-width spacing on curved or merging road segments.

Controller behavior: The ego vehicle is controlled using a classical longitudinal PID controller tracking a target speed. The controller operates purely reactively based on velocity tracking error and does not incorporate predictive gap estimation or speed-adaptive look-ahead. Fixed controller parameters are maintained across experiments to isolate the impact of interaction timing and ego speed on control robustness.

Parameter configuration: The scenario exposes several key parameters that control interaction intensity and difficulty, including ego target speed, NPC cruising speed, initial longitudinal gap (d_0), TTC triggering threshold (`cut_at_ttc`), and lane-change duration.

In our preliminary evaluation, the ego target speed was varied across multiple regimes (e.g., 70–90 km/h) to investigate controller sensitivity to closing velocity. The NPC speed was fixed at 60 km/h to create controlled relative-speed conditions. The initial longitudinal gap was set to 100–200 m depending on the experiment to ensure that TTC-triggered events occur within the observable simulation window.

The TTC threshold (`cut_at_ttc`) directly determines interaction timing. Larger thresholds trigger earlier cut-in maneuvers with larger safety margins, while smaller thresholds generate more aggressive and safety-critical scenarios. By sweeping this parameter in batch evaluation, we systematically control scenario difficulty.

The lane-change duration controls lateral smoothness and interaction abruptness. A fixed duration (e.g., 6.5 s) was used to maintain consistent lateral dynamics across experiments.

All controller gains were kept fixed throughout the evaluation to isolate the effect of speed and interaction timing on longitudinal robustness. This design ensures that observed performance differences arise from scenario parameters rather than retuning of control gains.

V. BASELINE RESULTS

Before evaluating the baseline, we hypothesize that the normal ego policy will favor safety over efficiency, producing higher safety margins but longer completion times due to

earlier braking and larger gap thresholds. In contrast, we expect the aggressive ego policy to complete maneuvers faster in normal traffic, but to lose robustness more quickly in aggressive traffic because of its smaller buffers and later interventions. Across scenarios, we expect the unprotected left turn to be more difficult than the highway merge, since it requires commit-or-yield decisions against oncoming cross-traffic in a compact conflict zone. By comparison, we expect highway merging to reward moderate assertiveness, since successful merging often depends on active speed matching and timely gap entry rather than purely conservative yielding.

For our preliminary results, we ran the describes simulation setups for our three scenarios.

A. Scenario 1 and 2: Left Turn/Highway Merge

For scenario 1 and 2, each combination of ego vehicle behavior (aggressive/normal) with traffic behavior (aggressive/normal) had 100 epochs. Figure 2 indicates the success rate of each combination in both scenarios. Figure 3 compares the jerk and time of completion for both scenes. The jerk depicted is the 90th percentile of jerk’s recorded for successful runs. The labels indicated are as follows:

- NN = normal ego / normal traffic
- AN = aggressive ego / normal traffic
- AA = aggressive ego / aggressive traffic
- NA = normal ego / aggressive traffic

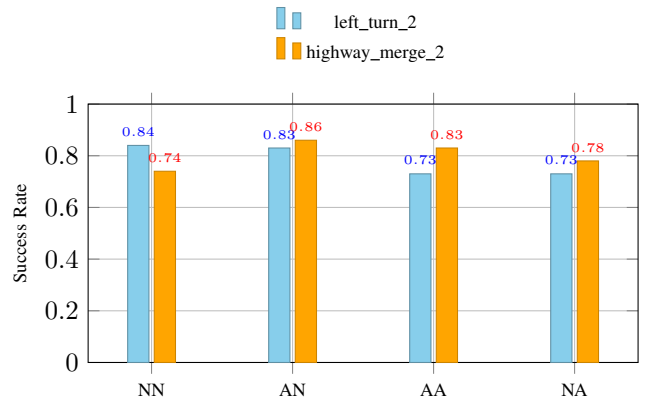


Fig. 2. Success-rate comparison (NN, AN, AA, NA).

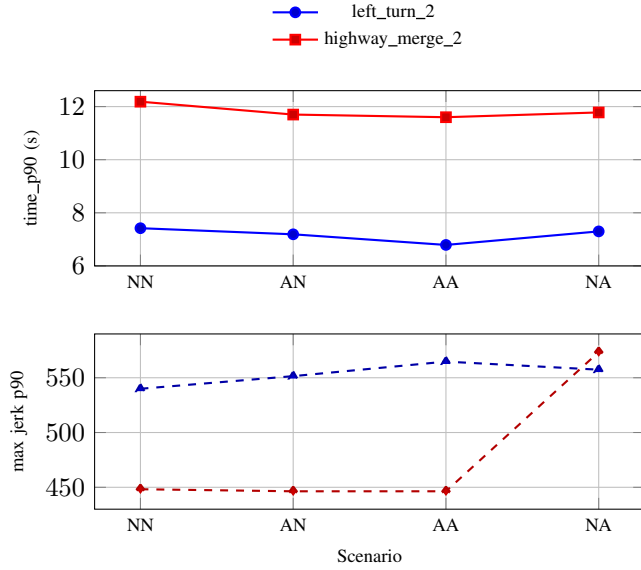


Fig. 3. P90 completion time and P90 max jerk for successful episodes.

B. Scenario 3: Cut-In Results

To evaluate the PID baseline under controlled cut-in conditions, we conducted experiments by varying ego speed while keeping all other parameters fixed. The NPC speed was set to 60 km/h, the initial longitudinal gap was 100 m, the TTC triggering threshold was 15.5 s, and the lane-change duration was fixed at 6.5 s.

Table II summarizes representative outcomes.

Ego Speed (km/h)	Collision	Observed Behavior
70	No	Safe deceleration and avoidance
90	Yes	Collision after cut-in

TABLE II
CUT-IN SCENARIO PERFORMANCE UNDER IDENTICAL PARAMETERS.

At 70 km/h, the ego vehicle successfully maintains a positive longitudinal safety margin and avoids collision. However, at 90 km/h, the same PID configuration results in collision despite identical TTC triggering conditions. This indicates strong sensitivity of the baseline controller to closing velocity.

Further inspection of control traces reveals that braking is initiated reactively after TTC threshold crossing, and deceleration ramps up gradually rather than aggressively. As a result, the longitudinal gap collapses before sufficient braking authority is established at higher speeds.

VI. BASELINES DISCUSSION

A. Key Findings

The baseline results show that the baseline stack is functional and reproducible across the three target interaction scenarios. In left-turn and merge settings, performance is strongly conditioned on behavior mode and traffic aggressiveness. The cut-in study shows a clear speed sensitivity: under identical TTC-triggered cut-in conditions, the PID baseline avoids collision at 70 km/h but fails at 90 km/h. This indicates

that current robustness is limited by interaction intensity rather than implementation instability.

B. Interpretation

The observed trends are consistent with a reactive controller architecture. The longitudinal PID policy responds only after threshold crossing and does not explicitly reason about future gap evolution. At higher closing speeds, delay in effective braking causes rapid safety-margin collapse. More aggressive behavior settings increase this effect by reducing proximity buffers and delaying intervention. The aggregate results suggest a tradeoff: maintaining progress under aggressive interactions currently degrades safety and comfort margins.

1) *Left Turn/ Highway Merge Analysis:* As seen in Figure 2, the ego vehicle has a high success rate in normal traffic comparatively to aggressive traffic for left turns. This implies the reactive controller does not react fast enough to avoid collision. On the other hand, for the highway merge scenario, the aggressive ego vehicle has a better success rate from the normal vehicle. This result may indicate lower risk margins are better for merging on the highway. Comparatively, the left turn scenario looks to be more complex than the highway merge since there is relatively lower success rate. Figure 3 depicts the stable completion time in both scenarios. Lastly, the highway scenario seems to have a higher jerk value for the normal ego in aggressive traffic. This most likely depicts the scenario where the vehicle must speed up once merged to match the traffic speed and avoid collision.

C. Limitations

This phase has several constraints. First, controller gains are fixed across all conditions, so we do not yet isolate whether failures are due to control structure or parameter choice. Second, scenario coverage is limited to a small set of maps and scripted interaction patterns. Third, evaluation is performed with simulator state access and does not yet include perception noise or latency in the full loop. Finally, we have not yet run broad statistical tests across large seed sweeps.

D. Simulation Constraints

The main external threat is simulator-to-real transfer; successful simulation behavior may not generalize to real traffic dynamics. Internal validity depends on scenario parameter choices (e.g., TTC trigger and lane-change duration), which can shift observed failure boundaries. There is also risk of seed- or map-specific bias in reported metrics. To reduce these threats, later phases should expand map diversity, increase seed counts, and report confidence intervals for key metrics.

E. Why Learning

As mentioned, the results indicate low reaction speeds and not enough reasoning as the main factor to crashing. A rule-based planner with fixed thresholds can handle many nominal cases, but its behavior becomes brittle as interaction timing tightens or traffic becomes more aggressive. This motivates a

learning-based planner that can map complex scene observations directly to short-horizon trajectories, allowing the ego vehicle to respond to interaction structure in a more adaptive and anticipatory way than a hand-designed rule set.

We predict a learning-based algorithm will improve performance all scenarios by learning better gap selection, smoother speed adjustment, and more context-sensitive yield-versus-go behavior. In the left-turn case, a learned planner should be able to predict on coming spacing and react accordingly. In the highway-merge case, it should better coordinate longitudinal timing with gap spacing from behind, reducing hesitation in feasible merges while avoiding overly aggressive entries. For the abrupt cut-in scenario, a learned model should react to the behavior of the nearby vehicle, assess open lanes, and change lanes accordingly. To study these improvements while keeping the rest of the stack unchanged, we replace the rule-based local planning layer with a reinforcement learning policy that outputs short-horizon trajectories and retain the same PID tracker used in the baseline. Among reinforcement learning methods, PPO is a practical choice since it provides stable on-policy training in continuous, closed-loop simulation and is well suited for optimizing behavior under a shared reward that encodes safety, comfort, and task completion.

VII. PPO PLANNER

Our PPO planner is implemented as a short-horizon local planner on top of a fixed global route backbone. At the beginning of each episode, a CARLA global route planner generates an initial route using A* search. This route is treated as a stable long-horizon reference for the policy. However, the global routes is not strictly fixed. If the ego vehicle deviates significantly from the route (beyond a predefined threshold of approximately 8 meters), a new global route is recomputed from the current vehicle position to the destination. This rerouting mechanism acts as a recovery strategy to maintain the consistency of the route.

On top of this global route, PPO re-plans a short local trajectory segment at every simulation step. The resulting local trajectory is then tracked by the same low-level CARLA VehiclePIDController, which keeps the long-horizon navigation problem stable while allowing the policy to react to nearby traffic online. This design separates planning from control, and enabling a controlled comparison between fixed trajectory following and adaptive short-horizon trajectory re-planning under identical control conditions.

A. Observation Design

The observation is designed to capture three complementary aspects of the driving tasks: (i) ego state and route-following status, (ii) gap-level interaction cues for maneuver timing, and (iii) local traffic geometry. The full observation has dimension $12 + 10 + 8N$, where N is the number of nearby vehicles (usually $N = 4$ in our experiments).

The first 12 dimensions encode ego motion and route-tracking information, including speed, distance to goal, heading and lateral errors relative to the route, and tracking state

with respect to the local trajectory. These features ensure stable progress along the global route while maintaining accurate local tracking.

To model interaction explicitly, we include 10 merge-specific features that summarize the structure of available gaps, such as front and rear distances, relative speeds, and TTC-based urgency, along with estimated merge timing. This representation provides the policy with direct access to gap feasibility, rather than requiring it to infer interaction structure from raw geometric states.

For each of up to N nearby vehicles, we include ego-centric features describing relative position, relative velocity and distance. This preserves local traffic geometry and enables the policy to reason about short-horizon interactions with surrounding vehicles.

The observation is structured to decouple route-following from interaction reasoning: route-related features guide long-horizon progress, while gap-level and traffic-level features support decision-making under dynamic traffic conditions.

B. Action Space

The PPO model uses a continuous action space with 5 normalized outputs. Instead of directly providing motor commands, the model produces a local trajectory that is tracked with CARLA PID low level control. This method helps focus the problem specifically on the planning rather than having the model learn controls as well. The five outputs are three lateral offsets, one heading reconnection point bias, and one target speed value.

The first three components indicate the local trajectory creation. The lateral offsets are three anchor points placed on the global trajectory (A* waypoints) approximately 8m, 18m, and 30m ahead of the ego vehicle. The offsets indicate a lateral shift of the anchors by maximum magnitude of 1.0m from the global trajectory. The fourth component in the action space, the reconnecting heading bias, affects the direction of the local trajectory as it connects back to the global trajectory. Internally, the environment converts these four action parameters into a smooth trajectory by interpolating between the three lateral offsets and reconnection point with a Catmull-Rom spline (Figure 4). Lastly, the fifth component controls the speed of the vehicle to decide the yield-and-go behavior of the vehicle.



Fig. 4. Trajectory creation method from three lateral offset points and one heading reconnection point bias. *Made with Gemini

C. Reward Function

The reward function was designed to encourage goal progression while maintaining safety and comfort. The main

positive term is a progress reward that is computed from the distance to the goal position at each step, encouraging progress towards the end goal. Furthermore, there is a per-step penalty as the episode progresses. This prevents a situation where the model maximizes reward by staying still and avoiding all collisions. Around the progress term, the environment applies several penalties that reflect comfort, safety, and path quality, together with episode terminal rewards.

For comfort and tracking quality, the code penalizes large steering magnitude, braking effort, deviation between the vehicle’s current speed and commanded target speed, and abrupt steering changes between time steps. These values enforce a smoother and stable trajectory as opposed to over oscillation and harsh behavior. Additionally, a lane penalty is applied to ensure the trajectory sticks closely to the global route and only deviates when necessary to safely reach its goal.

For traffic safety, there are penalties that compound whenever the ego vehicle enters a safety bubble of nearby vehicles. There is also a penalty incurred when the ego vehicle enters a low time-to-collision (TTC) situation. For increased safety, there is a path-overlap penalty that penalize local trajectory that may lead to a collision. With these terms, we expect the model to plan trajectories that ensure safety, prevent collisions, and encourage the model to “find the gap”.

Finally, the terminal reward strongly shapes episode outcomes. Reaching the goal produces a large positive reward, while collisions and timeouts incur substantial penalties. The code also applies an additional penalty for certain early crashes associated with poor lane placement, further discouraging obviously invalid or unstable maneuvers. Overall, the reward can be interpreted as a weighted balance of progress, comfort, and interaction-aware safety: the agent is encouraged to finish the task efficiently, but only by using smooth trajectories and choosing traffic gaps that are safe enough to commit to.

VIII. RESULTS

We evaluated the PPO-based planner on the highway merge scenario in closed-loop CARLA simulation using the same fixed low-level PID tracker as in the baseline system. This lets us attribute changes in performance primarily to the planner rather than to differences in vehicle control. The PPO policy was trained to generate a short-horizon local trajectory segment and target speed at each simulator step while remaining anchored to the global route. We then measured performance using the same scenario-level metrics used throughout the project, with emphasis on safety, efficiency, and comfort.

TABLE III
HIGHWAY MERGE BASELINE VS. PPO.

Method	Success Rate	Crash Rate	P90 Time (s)	P90 Jerk (m/s^3)
Baseline (Agg.)	83%	17%	11.6	446.354
PPO	86%	10%	17.4	547.138

The tuned PPO planner achieved an 86% success rate on the highway merge task. For successful episodes, the

90th-percentile completion time was 17.400 s, and the 90th-percentile maximum jerk was 547.137 m/s^3 . These results show that the PPO planner is capable of completing the merge reliably in a large majority of trials while maintaining a measurable closed-loop performance profile that can be compared directly against the classical baseline. Relative to our earlier PPO iterations, this updated result reflects a meaningful improvement in merge completion reliability and demonstrates that the RL pipeline is not only functional, but also responsive to tuning.

A key finding from PPO tuning was that the policy benefited from being framed as a local trajectory generator rather than as a selector over predefined route waypoints. Keeping the global route fixed while allowing PPO to reshape only the short-horizon local path made the task more stable and helped the agent learn interaction-aware behavior in dense traffic. We also found that reward design mattered significantly: stronger shaping around progress, collision avoidance, route adherence, and anti-idling behavior helped reduce failure cases where the ego vehicle hesitated excessively or became stuck waiting for traffic to fully clear. In addition, including nearby-vehicle features and route-relative state in the observation improved the agent’s ability to react to surrounding traffic during the merge.

Despite the improved success rate, the tuned PPO policy still exhibits limitations. The jerk metric indicates that some successful merges remain abrupt, suggesting that the learned planner can still rely on aggressive corrective actions in difficult interactions. Similarly, while the completion-time metric is improved, it indicates that some successful merges are still slower than desired. This suggests that the current policy has learned the merge objective, but still does not consistently achieve the best balance between decisiveness and smoothness.

Overall, the PPO results are encouraging because they demonstrate that a learned planner can be trained and evaluated end-to-end within our closed-loop framework for interactive highway merging. The strongest takeaway from this phase is that the system now supports meaningful policy iteration: planner changes produce measurable differences in success rate, completion time, and comfort, making the framework suitable for continued RL improvement. Future PPO tuning will focus on reducing sharp maneuvers, improving merge timing, and narrowing the gap between successful completion and comfortable execution.

IX. CONCLUSION AND FUTURE WORK

Phase 1 established a reproducible closed-loop simulation framework for evaluating autonomous vehicle behavior across three safety-critical interaction scenarios: unprotected left turns, highway merges, and abrupt cut-ins. Using CARLA as the simulation platform and a hierarchical PID-based baseline comprising global A* routing, local waypoint planning, and separate longitudinal and lateral PID controllers, we showed that classical reactive control is functional and measurable, but carries clear robustness limitations under aggressive interaction conditions.

For the Phase 1 baseline, left-turn and highway merge success rates across behavior-traffic combinations ranged from 73% to 86%, with aggressive ego and traffic pairings producing the highest collision rates and comfort degradation. For the cut-in scenario, the baseline successfully avoided collision at 70 km/h but failed at 90 km/h under identical TTC-triggered conditions, highlighting a fundamental speed sensitivity in purely reactive control: at higher closing velocities, the delay between threshold crossing and effective braking authority is too large to prevent gap collapse. Across all three scenarios, a consistent pattern emerged: the baseline controller responds only to instantaneous signals and does not reason over future interaction trajectories. This reactive structure becomes increasingly brittle as interaction intensity grows, whether through higher ego speeds, more aggressive NPC behavior, or tighter TTC triggering thresholds. The smooth cosine-based NPC lane-change model developed for the cut-in scenario also confirmed that physically consistent, reproducible disturbances can be generated deterministically, providing a solid foundation for controlled evaluation.

Phase 2 extended this framework with a PPO-based planner for the highway merge scenario while keeping the PID tracker fixed. Instead of following a rule-based local waypoint planner, the learned planner generates a short-horizon local trajectory segment and target speed at each simulator step while remaining anchored to a global route. With the tuned PPO planner, the highway merge policy achieved an 86% success rate, with a P90 completion time of 17.400 s for successful episodes and a P90 maximum jerk of 547.137 m/s³. These results show that the RL pipeline is now working end-to-end in closed loop and that the learned planner can complete the merge in a large majority of trials.

Several findings from PPO tuning were important. First, PPO performed better when framed as a local trajectory generator rather than as a selector over predefined route waypoints. Keeping the long-horizon route fixed improved task stability, while allowing the policy to reshape the near-term path made it more responsive to surrounding traffic during the merge. Second, reward tuning had a strong effect on behavior. Stronger incentives for progress, collision avoidance, route adherence, and reduced idling improved merge reliability and reduced failure modes where the ego vehicle hesitated too long or became stuck. At the same time, the jerk metric shows that some successful merges are still abrupt, indicating that the current policy does not yet consistently balance safety, smoothness, and decisiveness.

While the PPO model is promising for the highway merge scenario, it is not versatile enough to be applied to the left turn and cut-in scenarios. This limitation is likely due to the complex reward function and biased training. Future work would need to be done to get a single model that performs well for all three scenarios.

Overall, this work now provides both a reproducible classical baseline and a functional RL extension within the same closed-loop CARLA framework. The main next steps are to further improve PPO smoothness and merge timing, expand

evaluation across additional seeds and traffic settings, and extend learned planning to the other interactive scenarios. Longer term, this framework can support more systematic study of how learned planners trade off safety, comfort, and task completion under increasingly realistic interaction conditions. For phase 3, we intend to analyze these results with more realistic data. We will adjust the ground truth data with false positives, false negatives, gaussian noise and test if a learning algorithm trained on ground truth data versus false data is more robust in realistic situations.

REFERENCES

- [1] K. Chitta *et al.*, “Transfuser: Imitation with transformer-based sensor fusion for autonomous driving,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 11, pp. 12 878–12 895, 2023.
- [2] A. Dosovitskiy *et al.*, “Carla: An open urban driving simulator,” 2017.
- [3] C. Urmson *et al.*, “Autonomous driving in urban environments: Boss and the urban challenge,” *Journal of field Robotics*, vol. 25, no. 8, pp. 425–466, 2008.
- [4] B. Paden *et al.*, “A survey of motion planning and control techniques for self-driving urban vehicles,” 2016.
- [5] B. Varma, N. Swamy, and S. Mukherjee, “Trajectory tracking of autonomous vehicles using different control techniques(pid vs lqr vs mpc),” *IEEE Xplore*, 2020.
- [6] L. Wei *et al.*, “Autonomous driving strategies at intersections: Scenarios, state-of-the-art, and future outlooks,” *CoRR*, vol. abs/2106.13052, 2021. [Online]. Available: <https://arxiv.org/abs/2106.13052>
- [7] C. Hubmann *et al.*, “A pomdp maneuver planner for occlusions in urban scenarios,” in *2019 IEEE Intelligent Vehicles Symposium (IV)*, 2019, pp. 2172–2179.
- [8] Z. Qiao *et al.*, “Pomdp and hierarchical options mdp with continuous actions for autonomous driving at intersections,” in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*, 2018, pp. 2377–2382.
- [9] Z. Zhaojun *et al.*, “Lane change trajectory planning based on improved model predictive control with artificial potential field for autonomous vehicles in medium-high speed scenarios,” *Engineering Applications of Artificial Intelligence*, vol. 150, p. 110601, 2025.
- [10] M. Bansal, A. Krizhevsky, and A. Ogale, “Chauffeurnet: Learning to drive by imitating the best and synthesizing the worst,” 2018. [Online]. Available: <https://arxiv.org/abs/1812.03079>
- [11] D. Chen *et al.*, “Learning by cheating,” 2019. [Online]. Available: <https://arxiv.org/abs/1912.12294>